

Ayoub Sabri

BA35134

Section 7591

### Question 1:

#### 1. formulas

**Steps:**  $h(n)$  = number of positions where the current word and goal have different letters

**Scrabble:**  $h(n)$  = sum of SCRABBLE\_VALUES[goal[i]] for every position i where current word and goal have different letters

**Frequency**  $h(n)$  = number of mismatches + dictionary[goal], where num mismatches is the number of positions where the current word and goal differ, if num\_mismatches is 0,  $h(n)$  = 0

#### 2. lower-bounds

**steps:** since one action changes only one letter, every mismatched position needs at least one more action to fix it. so number of mismatches = the smallest number possible of moves still needed to reach goal.

**Scrabble:** whatever sequence of moves we need, position i has to end up with goal[i]. so somewhere in the remaining path, an action must place goal[i] at i, which will cost exactly SCRABBLE\_VALUES[goal[i]] which gives the minimum unavoidable cost still needed over every mismatched position.

**Frequency:** same reasoning as steps, num mismatched positions means at least num more actions are needed and every action costs at least 1 (1 + rarity)

#### 3. Overestimate

**Steps:** Each action changes one letter, so it can fix at most one mismatch. Which means the true number of remaining moves is at least equal to the num of mismatches, the heuristic never claims more remaining cost than what's required

**Scrabble:** each mismatched position only adds the cost of that one action required to place the correct letter (which costs `scrabble_values[goal[i]]`) never exceeds the true cost

**Frequency:** every piece of the bound is guaranteed, we cant finish in fewer moves and every move costs  $1 + \text{rarity}$ , with end goal costing exactly  $1 + \text{dictionary}[\text{goal}]$  so it could never overestimate the cost.

### Question 2:

| Word pair    | Cost measure | UCS cost | UCS nodes expanded | A* cost | A* nodes expanded | Reduction |
|--------------|--------------|----------|--------------------|---------|-------------------|-----------|
| WHY -> ASK   | steps        | 5        | 773                | 5       | 16                | 97.93     |
| WHY -> ASK   | Scrabble     | 9        | 765                | 9       | 13                | 98.3      |
| WHY -> ASK   | Frequency    | 38.86    | 808                | 38.86   | 634               | 21.53     |
| BEAR-> DUCK  | Steps        | 4        | 691                | 4       | 6                 | 99.1      |
| BEAR-> DUCK  | Scrabble     | 11       | 2981               | 11      | 6                 | 99.8      |
| BEAR -> DUCK | Frequency    | 38.24    | 1084               | 38.24   | 302               | 72.14     |

1. Yes, in every run UCS and A\* found solutions with identical costs this makes sense since our heuristic is admissible. A\* just finds it faster by expanding fewer nodes.
2. For both WHY->ASK and BEAR->DUCK scrabble gave the largest reduction (98.3 and 99.8)
3. Frequency was clearly the least informative for both word pairs, only 21.53% and 72.14% reduction
4. Steps and scrabble heuristics look closely at which letters are wrong and use that to guide the search so A\* can tell right away which paths are actually closer to the goal which is why it skips almost all the extra searching UCS does. The frequency heuristic only looks at how rare the final goal is, not the letters along the way so it can't tell good paths from bad ones

### Question 3:

**1. Steps: yes, scrabble: no, frequency: no**

**2. Steps:**

- Always equal, the action itself is reversible and under steps, every action costs exactly 1 regardless of the letters involved because it doesn't care about which letter was swapped only how many swaps happened.

Scrabble:

- Not always equal, the action is still reversible but the cost depends on which specific letter get placed and going in the opposite direction places different letters thus different values

Frequency:

- Not always equal, same idea as scrabble but  $W1 \rightarrow W2$  costs  $1 + \text{rarity } W2$  while  $W2 \rightarrow W1$  costs  $1 + \text{rarity } W1$ , different rarity values means different costs going backwards.

**3. Scrabble counterexample:**

dog  $\rightarrow$  dot changes the last letter from g to t, the cost is `scrabble_values[t] = 1`, if we reverse it dot  $\rightarrow$  dog changes t to g cost is `scrabble_values[g] = 2`

**frequency counterexample:**

same example, for dog  $\rightarrow$  dot cost would be  $1 + \text{rarity}(\text{dot})$ , while dot  $\rightarrow$  dog cost would be  $1 + \text{rarity}(\text{dog})$  since they are different words their rarity would certainly be different

in all 3 cases, the action itself is always reversible, what differs is whether undoing an action is same as doing it. only steps measure treats both directions the same while scrabble and frequency both depend on which specific word or letter is entered which have different costs depending on the direction

### Question 4 :

**1. I used Claude-opus-5-5**

**Prompt:** In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes

exactly one letter. Under the frequency objective, entering word  $w$  costs  $1 + \text{rarity}(w)$ , where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic  $h_0(n)$  is the number of positions in which state  $n$  differs from the goal. Propose

a rarity-aware heuristic intended to be more informative than  $h_0$  while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as  $h_0$  for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

**Answer:** Proposed heuristic:  $h_1$

Notation

- $g$  is the goal word and  $L = |g|$ .
- $D_L$  is the set of dictionary words of length  $L$ .
- $r_{\min} = \min_{w \in D_L} \text{rarity}(w)$ . This is computed once per problem, and  $r_{\min} \geq 0$ .
- $h_0(n)$  is the Hamming distance from  $n$  to  $g$ .

Definition

- $h_1(n) = 0$  if  $n = g$ .
- Otherwise,  $h_1(n) = h_0(n) + \text{rarity}(g) + (h_0(n) - 1) \cdot r_{\min}$ .

In words: each step costs at least 1. The last word entered is always the goal, so its rarity is always paid. Every other word entered costs at least the cheapest rarity in the dictionary.

Why it is admissible

Take any state  $n \neq g$  and any legal path  $n = w_0 \rightarrow w_1 \rightarrow \dots \rightarrow w_k = g$ .

1. The path has at least  $h_0(n)$  steps. Each action changes one letter, so it can reduce the number of mismatched positions by at most 1. That gives  $k \geq h_0(n)$ .
2. Split the path's cost. The cost is the sum of  $(1 + \text{rarity}(w_i))$  for  $i = 1$  to  $k$ . Separate out the final step, which enters  $g$ :

cost =  $k + \text{rarity}(g) + (\text{sum of } \text{rarity}(w_i) \text{ for } i = 1 \text{ to } k-1)$ .

3. Bound each piece from below. There are  $k - 1$  intermediate words. Each is in the dictionary, so each has  $\text{rarity} \geq r_{\min}$ . Using  $k \geq h_0(n)$ :

cost  $\geq k + \text{rarity}(g) + (k - 1) \cdot r_{\min} \geq h_0(n) + \text{rarity}(g) + (h_0(n) - 1) \cdot r_{\min} = h_1(n)$ .

This holds for every path, including the optimal one, so  $h_1(n) \leq h^*(n)$ . At the goal,  $h_1 = 0 = h^*$ . If no path exists,  $h^* = \infty$ , so the bound holds trivially.

It is also consistent. For an edge  $n \rightarrow m$  with cost  $1 + \text{rarity}(m)$ :

- If  $m = g$ , then  $h_0(n) = 1$ , so  $h_1(n) = 1 + \text{rarity}(g)$ . This equals the edge cost plus  $h_1(g) = 0$ .
- If  $m \neq g$ , then  $h_1(n) - h_1(m) = (1 + r_{\min})(h_0(n) - h_0(m)) \leq 1 + r_{\min} \leq 1 + \text{rarity}(m)$ .

So  $A^*$  using  $h_1$  never has to reopen a node.

Why it is more informative than  $h_0$

At least as large everywhere.  $\text{Rarity}(g) \geq 0$  and  $(h_0 - 1) \cdot r_{\min} \geq 0$  whenever  $n \neq g$ . So  $h_1(n) \geq h_0(n)$  for every state, with equality at the goal.

Strictly larger somewhere. If  $\text{rarity}(g) > 0$ , then  $h_1 > h_0$  for every non-goal state. The same holds for every state with  $h_0 \geq 2$  when  $r_{\min} > 0$ . For example:

- Goal "cat" has rarity 3.
- State "cot" has  $h_0 = 1$ .
- $h_1 = 1 + 3 = 4$ , which is exactly the optimal cost.
- $h_0$  underestimates it by 3.

$h_1$  adds the goal's rarity to every non-goal state, where  $h_0$  ignores rarity altogether.

Caveat. If every rarity is 0, the problem reduces to unit costs and  $h_1 = h_0$ . The strict improvement needs at least some positive rarity: either the goal's rarity or the dictionary minimum.

Optional refinement (still admissible)

When  $h_0(n) \geq 2$ , the second-to-last word on any path differs from  $g$  in exactly one letter, and it cannot be  $n$ . So one of the intermediate charges can be tightened:

- Let  $r_1 = \min \text{rarity}$  over dictionary words at Hamming distance 1 from  $g$ .
- For  $h_0(n) \geq 2$ , replace one  $r_{\min}$  term with  $r_1$ :

$$h_1'(n) = h_0(n) + \text{rarity}(g) + r_1 + (h_0(n) - 2) \cdot r_{\min}.$$

Since  $r_1 \geq r_{\min}$ , this is at least as large as  $h_1$ . It can also be strictly larger than  $h_0$  in cases where  $\text{rarity}(g) = 0$  and  $r_{\min} = 0$ .

## 2. $H(1) = 0$

**$n == \text{goal}$**

**$h_1(n) = h_0(n) + \text{rarity}(\text{goal}) + (h_0(n) - 1) \cdot r_{\min}$**

**otherwise**

**the minimum of number of moves needed where the very last move costs  $1 + \text{rarity}(\text{goal})$  and every other move costs at least  $1 + r_{\min}$**

3.  $H_1$  is admissible and never overestimates the true remaining cost. If we take any state  $n \neq \text{goal}$  and any legal path from  $n$  to goal taking  $k$  actions and since every action fixes one mismatched letter,  $k$  must be at least  $h_0(n)$ . the total cost of this path under frequency is  $1 + \text{rarity}$  w over every word along the way. The last action

adds exactly  $1 + \text{rarity}(\text{goal})$ . Each remaining action lands on some other dictionary word of the same length as the goal so each word adds at least  $1 + r_{\min}$  ( $r_{\min}$  is the cheapest rarity word).

the total cost of any path is at least  $(1 + \text{rarity}(\text{goal})) + (k-1)(1 + r_{\min})$  and since this expression only grows as  $k$  grows, the smallest possible value it can take happens at the smallest legal  $k$  which is  $k = h_0(n)$ . substituting that into the expressions:

$$(1 + \text{rarity}(\text{goal})) + (h_0(n) - 1)(1 + r_{\min}) = h_0(n) + \text{rarity}(\text{goal}) + (h_0(n) - 1) \times r_{\min} = h_1(n)$$

Therefore each legal path costs at least  $h_1(n)$ . and since this holds for every possible path, It also holds for the optimal path so  $h_1(n) \leq h^*(n)$ .  $h_1$  never overestimates the true remaining cost therefore it is admissible

#### 4. N/A

| Word pair    | h0 cost | h0 nodes expanded | h1 cost | h1 nodes expanded | Reduction |
|--------------|---------|-------------------|---------|-------------------|-----------|
| WHY -> ASK   | 38.86   | 755               | 38.86   | 634               | 16.03     |
| BEAR -> DUCK | 38.24   | 720               | 38.24   | 257               | 64.31     |

The evidence supports using the  $h_1$  heuristic over the baseline  $h_0$ .  $H_1$  found the same optimal solution cost as  $h_0$  while expanding fewer nodes. Which aligns with the theoretical argument that  $h_1$  is more informative than  $h_0$ .  $H_1$  gives  $A^*$  a tighter and more useful estimate of remaining cost.

#### Generative AI disclosure:

- I used Claude as a tutor throughout implementation of dc.py. for the implementation i used it to explain AIMA's problem/node interface and python concepts that I was unfamiliar with and ones that I forgot the right syntax for such as string slicing and list manipulation but I wrote the code myself. I also used it to help phrase my reasoning and make sure I am including everything the questions ask for. For question 4 part 5, I used Claude to help write the python script used to test the proposed heuristic against  $h_0$